

Relatório: Trabalho de Paralelismo em C++

Grupo: Anna Flávia Guenta Tsurushima, Giovanna Beatriz Ramos, Gustavo Alexandro Pelissari, Igor Henrique Koga Vigato.

Descrição do benchmark escolhido e por que o grupo escolheu

Benchmark escolhido: Opção 3 — Conjunto de Mandelbrot.

Justificativa: O grupo optou por este benchmark por duas razões principais. Primeiramente, o problema produz um resultado visual (a imagem do fractal) que facilita a verificação da corretude do algoritmo em todas as versões. Em segundo lugar, o Conjunto de Mandelbrot apresenta um cenário clássico de **desbalanceamento de carga (load balancing)**. O custo computacional de cada pixel varia drasticamente: pontos pertencentes ao conjunto, ou muito próximos da borda, exigem as 1000 iterações (muito pesados), enquanto pixels distantes "escapam" em 2 ou 3 iterações (muito leves). Isso nos permitiu explorar na prática como as diferentes APIs (std::thread, OpenMP e TBB) lidam com a distribuição dinâmica de tarefas para otimizar o tempo de execução.

Tabela de tempos e speedup de cada versão

ATENÇÃO: O grupo deve rodar o código compilado com a flag `-O3` e substituir os valores [XX ms] abaixo pelos resultados reais gerados no terminal. O speedup é calculado como: $S = \frac{\text{Tempo Sequencial}}{\text{Tempo Paralelo}}$. Consideramos o uso de 4 threads no experimento.

Média de 4 threads usadas executando três vezes:

Versão	Tempo de Execução (ms)	Speedup (S)
Sequencial	986,833 ms	1.00x
std::thread	249,853 ms	3.95x
OpenMP	249,826 ms	3.95x
Intel TBB	254,786 ms	3.87x

8 threads usadas executando três vezes:

Versão	Tempo de Execução (ms)	Speedup (S)
--------	------------------------	-------------

Sequencial	986,832666667ms	1.00x
std::thread	125,111 ms	7.89x
OpenMP	125,607 ms	7.86x
Intel TBB	130,332 ms	7.57x

Análise crítica: por que o speedup foi o que foi? Quais fatores limitaram?

Os resultados de speedup mostraram que a paralelização foi altamente eficiente. Como o processamento de cada pixel é perfeitamente independente, a aplicação das técnicas resultou em tempos significativamente menores em relação à versão sequencial.

No entanto, a forma como o trabalho foi distribuído impactou diretamente a performance:

- **std::thread:** Ao invés de uma divisão estática da matriz em blocos (o que sobrecarregaria a thread que calculasse a área preta do fractal), utilizamos uma **Fila Dinâmica de Trabalho** através de um ponteiro atômico (`std::atomic<int> proximaLinha.fetch_add(1)`). Assim, as threads requisitavam as linhas sob demanda, mitigando o desbalanceamento de carga.
- **OpenMP:** Para garantir um comportamento equivalente e evitar que alguma thread ficasse presa calculando as áreas centrais enquanto as outras ficavam ociosas, foi crucial utilizar a diretiva `schedule(dynamic)`.
- **Intel TBB:** O uso do `tbb::parallel_for` com `blocked_range2d` obteve excelentes tempos devido ao algoritmo nativo de work-stealing do TBB, que equilibra a carga automaticamente entre os núcleos da CPU.

Fatores Limitantes: O speedup ideal (exatamente igual ao número de núcleos, ex: 4.0x para 4 threads) raramente é alcançado devido ao overhead na criação/junção das threads e também por um leve grau de contenção no acesso à variável atômica ou à fila do scheduler do OpenMP. Por fim, nas últimas linhas da imagem, é inevitável que algumas threads fiquem ociosas aguardando a finalização do processamento das tarefas remanescentes pelas outras threads.

Dificuldades encontradas e como foram resolvidas

1. Mapeamento de Coordenadas: Transformar os índices iterativos (x, y) da imagem (resolução 1920 x 1080) nas coordenadas reais contínuas do Plano Complexo exigidas pela matemática do fractal (eixo real entre -2.0 e 1.0; eixo imaginário entre -1.2 e 1.2).

- *Solução:* Utilizamos cálculo de interpolação linear (regra de 3) em cada iteração `for`.

2. Ciclo de Vida das Variáveis de Cálculo (z_x , z_y): Inicialmente, as variáveis de estado de cálculo dos pontos estavam sendo declaradas fora do laço interno do eixo X. Isso causava o acúmulo de estado de um pixel para o próximo.

- *Solução:* As variáveis foram escopadas para o interior do laço do cálculo de cada pixel, garantindo que o ponto iniciasse invariavelmente em $z_0 = 0 + 0i$.

3. Erros de Linkagem na Compilação (Linux): Ao tentar compilar o código utilizando apenas `clang` ou `g++` simples, nos deparamos com erros como `undefined reference`.

- *Solução:* Compreendemos que as bibliotecas externas não são incorporadas por padrão. Para a build ser bem-sucedida, inserimos as devidas bibliotecas na linha de comando: `g++ -std=c++17 -O3 main.cpp -o benchmark -fopenmp -ltbb -pthread`.